

INTERNSHIP PROJECT REPORT

TASKFLOW

A Full-Stack Task Management and Team Collaboration Platform

Submitted By:

Emaan Abid

Internship Organization:

FORTEX

Internship Project:

2026

Table of Contents

1. Project Overview	8
2. Problem Statement	8
3. Project Objectives	9
4. Scope of the Project.....	9
User Management	10
Project Management.....	10
Task Management	10
Collaboration	11
Analytics.....	11
Account and System Management.....	11
5. User Authentication	11
Signup.....	12
Login	12
Logout.....	13
Password Reset	13
6. Dashboard	14
Dashboard functionality	14
Team Members.....	15
Recent Activity.....	15
Project Progress.....	15
Deadlines	15
Task Completion Chart.....	15
Dashboard Search.....	15
7. Project Management	16
Project Creation.....	16
Project Updating	16
Project Deletion.....	16
Project Permissions	17
8. Task Management.....	17
Task CRUD.....	18

Task Assignment	18
Task Permissions	18
Project Owner	18
Assignee.....	18
9. Kanban Board	18
10. Drag-and-Drop Task Management	19
11. Task Creation.....	20
Floating Add Button	20
Column Add Buttons	20
12. Task Drawer	20
Save Task Loading State.....	20
Task Comments	21
Task Attachments	21
Task Activity Timeline	21
13. Activity Page	21
14. Search Functionality	22
15. Team Management	22
16. Invitation System.....	23
17. Notifications.....	23
Notification Features	23
18. User Profile Management	24
19. Account Management	25
20. Notification Management	26
21. Billing Management	26
Billing Features	27
Payment Method	27
22. Recycle Bin / Soft Delete.....	28
23. Backend Architecture.....	28
Middleware Layer	28
Controller Layer	29
Service Layer	29

Repository Layer	29
Global Error Handling	29
24. Error Handling	29
25. Database	30
Database Entity Relationships	30
Database ERD	31
26. Redis Caching	33
27. API Documentation	33
28. API Summary	33
API Request Flow	34
29. Environment Configuration	35
30. Security and Authentication	35
Authentication Flow	35
JWT Authentication	35
Authorization	36
Logout and Token Handling	36
Password Management	37
Two-Factor Authentication	37
Backend Validation	37
Secure Payment Metadata	37
31. Role-Based Access Control and Permissions	38
Project Owner	38
Assignee	38
Team Member	39
Permissions Matrix	39
Authorization Principle	39
32. Frontend Technology Stack	40
33. Backend Technology Stack	40
34. System Architecture	41
35. Complete Application Workflow	42
36. Functional Requirements	42

37.	Non-Functional Requirements	43
38.	Responsive Design	43
39.	External/Supporting Services.....	43
40.	Installation Requirements	43
	Recommended hardware	44
	Supported OS.....	44
41.	Local Installation.....	44
	Step 1: Clone Repository	44
	Step 2: Backend Setup.....	44
	Step 3: Frontend Setup	45
	Step 4: Running the Complete Application	45
	Step 5: API Documentation	45
42.	Deployment	46
43.	Live Application	46
44.	Testing and Validation	46
	Test Case Format.....	47
	Functional Test Cases	47
	Security Test Cases.....	49
	Performance Validation	50
	Deployment Validation	51
45.	Strengths of the System	51
	1. Centralized Management	51
	2. Visual Workflow.....	51
	3. Role-Based Access	51
	4. Collaboration.....	51
	5. Analytics	51
	6. Performance	52
	7. Maintainable Backend	52
	8. Production Deployment	52
	Limitations	52
46.	Implemented Features and Future Planned Features	52

Currently Implemented Features.....	52
Authentication and Account.....	52
Project Management.....	53
Task Management.....	53
Collaboration.....	54
Dashboard.....	54
Account and System.....	54
Deployment.....	54
Future Planned Features	55
Email Notifications and Email Invitations	55
Advanced Analytics.....	55
Calendar Integration	55
47. Conclusion	56

Table of Figures:

Figure 1: Signup page for new users to make an account	12
Figure 2: Login Page for the users that already have an account	13
Figure 3: Forgot Password page to reset the password	14
Figure 4: Dashboard Page showing the team members, recent activities, project progress, deadlines, task completion chart, and projects/task statistics.	16
Figure 5: Project drawer to create, update and delete a project.....	17
Figure 6: Project page showing the project and their tasks in the kanban	19
Figure 7: Task drawer to create, update, and delete a task, also to add comments and attachments to the task.....	21
Figure 8: Activity page showing all the activities of the tasks which are owned by the user, or they are part of that task	22
Figure 9: Team page where the user can invite a new team member and can view all the team members on this page.....	23
Figure 10: Notification icon on the dashboard page notifies about project and task changes and invitation status	24
Figure 11: Profile page of the user to update their profile	25
Figure 12: Account page to update user email or password, to enable 2FA, or delete their account permanently	25
Figure 13: Notification page to manage the notifications	26
Figure 14: Billing page to manage the bills and subscriptions.....	27
Figure 15: Recycle Bin page which shows the deleted projects and tasks that can be restore of permanently deleted	28
Figure 16:Entity Relationship Diagram of the TaskFlow database	32
Figure 17: The complete system architecture of the TaskFlow	41
Figure 18: User workflow of the TaskFlow.....	42

1. Project Overview

TaskFlow is a modern full-stack task management and team collaboration web application designed to help individuals and teams organize projects, manage tasks, monitor progress, and collaborate efficiently through a centralized platform.

The application provides a Kanban-style project management workflow where users can create projects, manage tasks, assign tasks to team members, update task statuses using drag-and-drop functionality, communicate through comments, attach files, monitor activities, receive notifications, and analyze project progress.

TaskFlow also provides account, profile, notification, billing, invitation, and activity-management functionality, making it more than a simple task manager. The application is designed as a complete project-management platform with role-based permissions, real-time-oriented dashboard updates, caching, authentication, automated database setup, and production deployment.

The system consists of two major components:

- **Frontend:** React-based user interface
- **Backend:** Node.js/Express REST API connected to PostgreSQL through Prisma ORM

The frontend and backend are deployed separately, with the frontend hosted on Vercel and the backend and the database hosted on Railway.

2. Problem Statement

Managing projects and tasks through scattered tools can make it difficult for teams to maintain a clear understanding of project progress, task ownership, deadlines, communication, and recent activities.

Common problems include:

- Difficulty organizing multiple projects
- Lack of centralized task management
- Poor visibility of task ownership
- Difficulty tracking project progress
- Lack of clear task statuses
- Communication being separated from tasks
- Difficulty tracking project activity
- Lack of centralized notifications
- Difficulty managing team members and invitations
- No clear role-based control over projects and tasks

- Difficulty monitoring deadlines

TaskFlow addresses these problems by providing a centralized platform that integrates projects, tasks, users, communication, notifications, activities, and account management into a single application.

3. Project Objectives

The main objectives of TaskFlow are:

1. To provide a centralized task and project management platform.
2. To allow users to create and manage multiple projects.
3. To provide a Kanban-based workflow for task management.
4. To allow project owners to assign tasks to team members.
5. To implement role-based permissions for projects and tasks.
6. To provide drag-and-drop task management.
7. To provide task comments and attachments.
8. To maintain activity history for projects and tasks.
9. To provide dashboard analytics and project statistics.
10. To provide centralized in-app notifications.
11. To provide user profile and account management.
12. To provide billing and subscription management.
13. To provide invitation functionality for collaboration.
14. To provide search functionality for tasks and dashboard information.
15. To implement secure authentication and authorization.
16. To provide automated database setup.
17. To standardize backend error handling.
18. To organize backend code using layered architecture.
19. To improve application performance using caching.
20. To deploy the complete application to production.

4. Scope of the Project

TaskFlow covers the following major areas:

User Management

- Signup
- Login
- Logout
- Password reset
- Profile management
- Profile image management
- Email management
- Password management
- Two-factor authentication
- Account deletion

Project Management

- Project creation
- Project viewing
- Project updating
- Project deletion
- Project progress tracking
- Project members
- Project permissions
- Soft deletion

Task Management

- Task creation
- Task viewing
- Task updating
- Task deletion
- Task assignment
- Task comments
- Task attachments
- Task activity timeline
- Task search

- Task status management
- Drag-and-drop

Collaboration

- Team members
- Invitations
- Comments
- Attachments
- Activity tracking
- Notifications

Analytics

- Project statistics
- Task statistics
- Completed task statistics
- Deadlines
- Project progress
- Task completion chart
- Recent activities

Account and System Management

- Notification preferences
- Billing
- Subscription plans
- Invoice information
- API documentation
- Automated database setup
- Error handling
- Caching

5. User Authentication

TaskFlow provides authentication functionality for protecting user accounts and application resources.

Signup

New users can create accounts through the signup page.

The authentication workflow includes:

User → Signup Form → Backend API → Validation → User Creation → Database → Account Created.

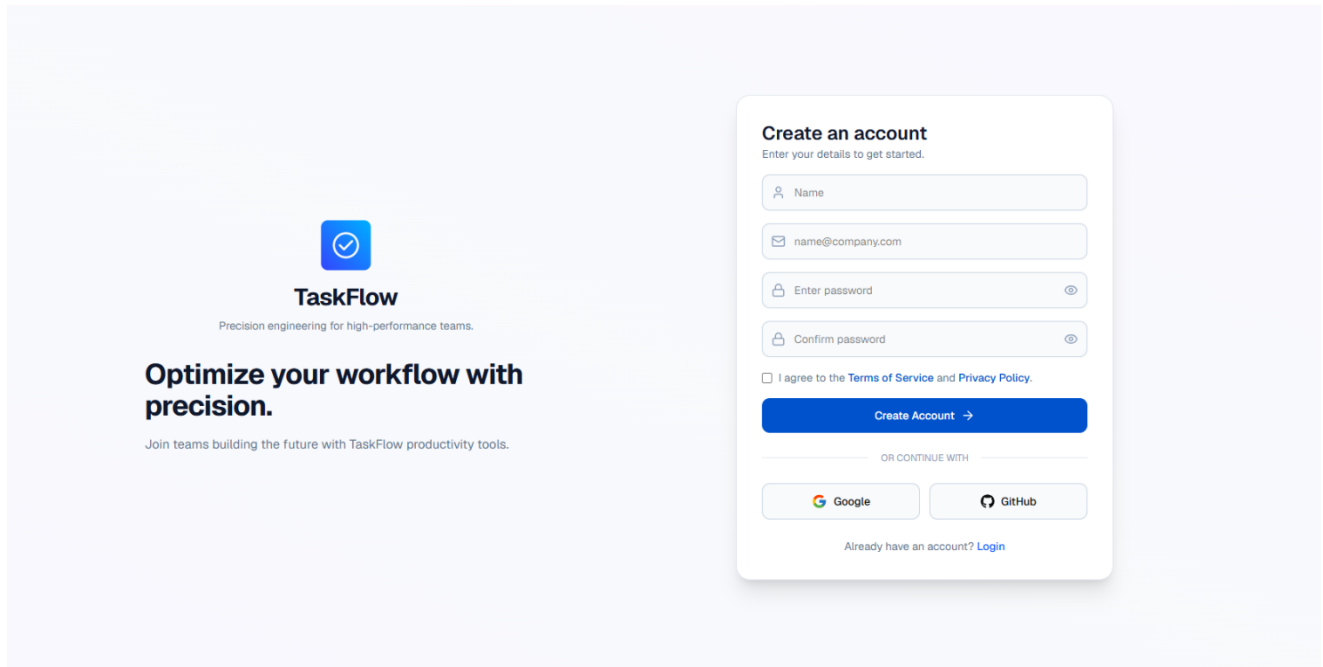
The image shows a web page for TaskFlow, a productivity tool. On the left, there is a blue square logo with a white checkmark, followed by the text "TaskFlow" and "Precision engineering for high-performance teams." Below this, a bold heading reads "Optimize your workflow with precision." and a subtext says "Join teams building the future with TaskFlow productivity tools." On the right, there is a white card titled "Create an account" with the subtitle "Enter your details to get started." The card contains four input fields: "Name", "Email" (with the placeholder "name@company.com"), "Enter password", and "Confirm password". Each password field has an eye icon to toggle visibility. Below the fields is a checkbox for "I agree to the Terms of Service and Privacy Policy." and a blue "Create Account" button with a right arrow. Underneath the button, it says "OR CONTINUE WITH" followed by two buttons for "Google" and "GitHub". At the bottom of the card, it asks "Already have an account? Login" with a link. The footer of the page includes "© 2026 TaskFlow Inc. All rights reserved." and links for "Privacy Policy", "Terms of Service", and "Status".

Figure 1: Signup page for new users to make an account

Login

Registered users can log into the application using their credentials.

The application uses JWT-based authentication for authenticated API requests.

The general workflow is:

Login → Credentials Verification → JWT Generation → Token Stored by Frontend → Authenticated API Requests.

The frontend sends the authentication token with protected API requests.

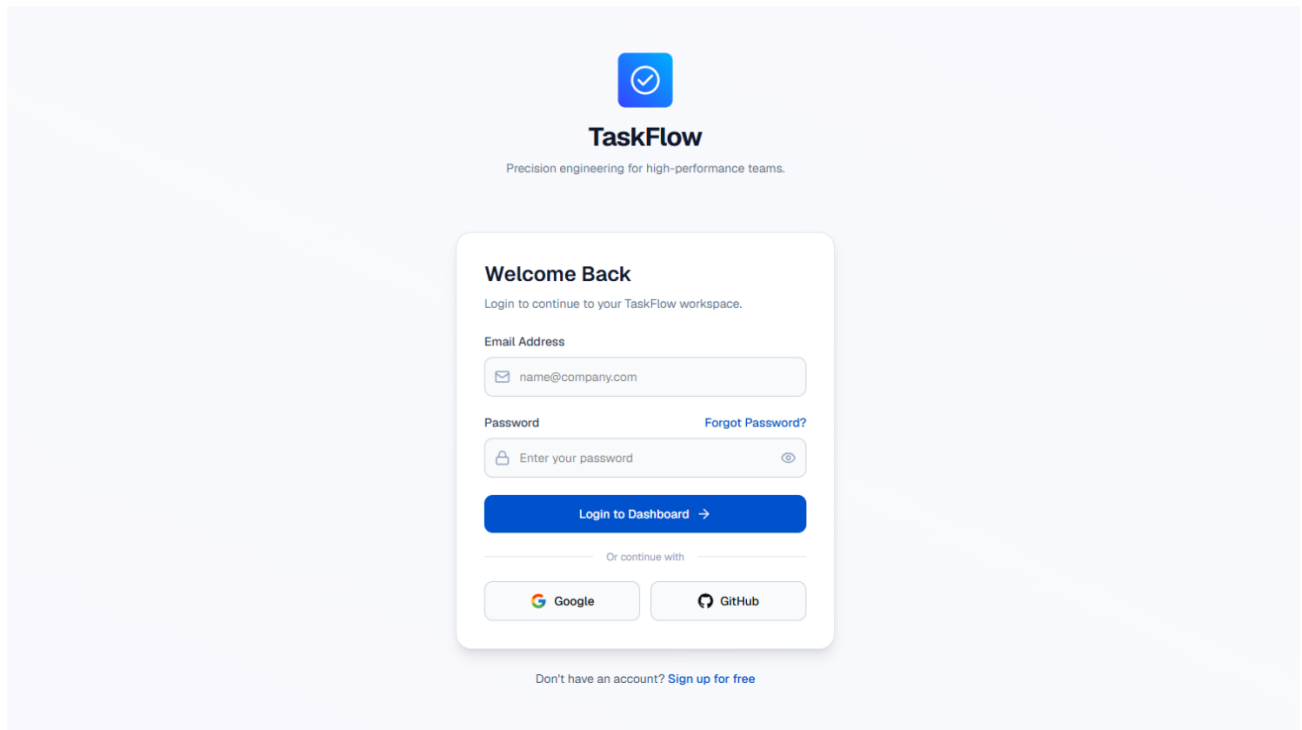


Figure 2: Login Page for the users that already have an account

Logout

Logout functionality has been implemented on both:

- Frontend
- Backend

This ensures that logout is not only a client-side operation but is also handled by the backend authentication system.

Password Reset

The application provides a Forgot Password feature.

Users can initiate password recovery from the login page and reset their password.

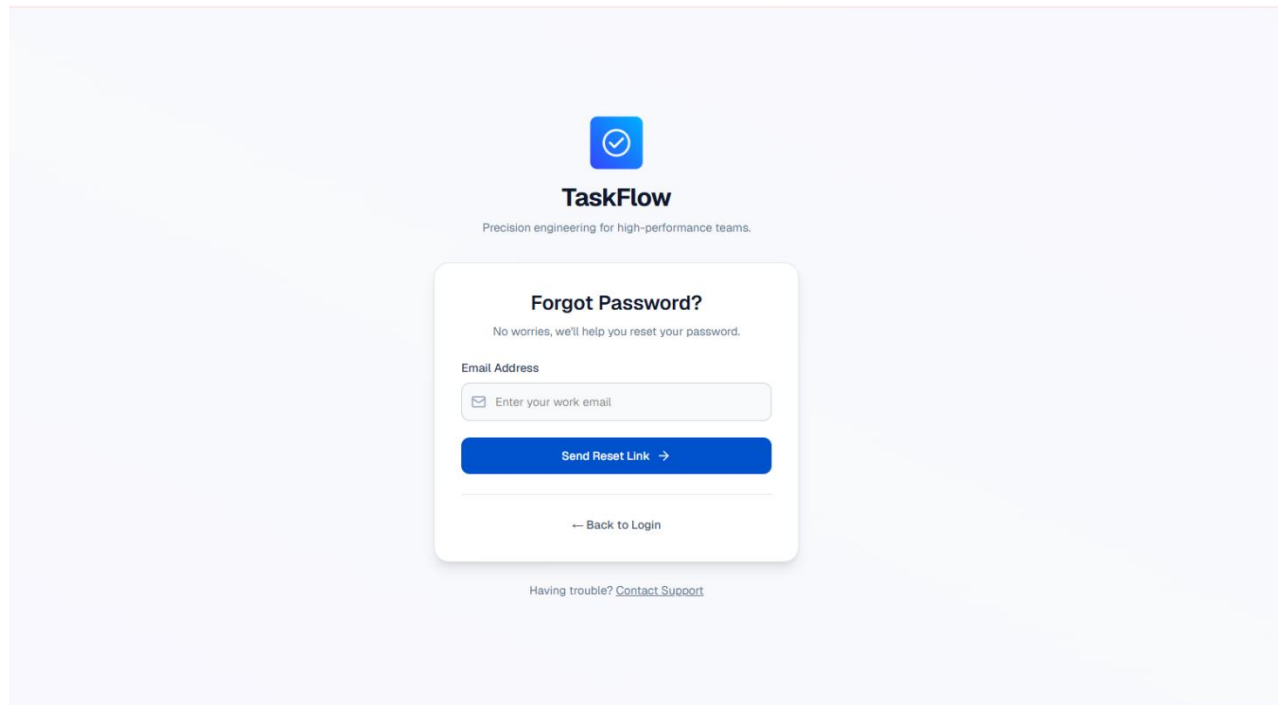


Figure 3: Forgot Password page to reset the password

6. Dashboard

The Dashboard acts as the central overview of the user's TaskFlow workspace.

It provides real-time-oriented information about projects, tasks, team members, deadlines, progress, and activities.

Dashboard functionality

The dashboard displays:

- Total projects
- Task statistics
- Completed tasks
- Team members
- Recent activities
- Project progress
- Upcoming deadlines
- Task completion chart
- Dashboard search

Team Members

The dashboard displays team members associated with the user's projects.

Users can click a team member:

- Name
- Profile image

to open that member's profile in read-only mode.

Recent Activity

The dashboard displays recent activity related to the user's projects and tasks.

This allows users to quickly identify recent changes without manually opening every project.

Project Progress

Project progress is represented using the number of completed tasks compared to the total number of tasks.

Deadlines

Upcoming task deadlines are displayed on the dashboard. The deadline section uses a scrollable view so that multiple upcoming deadlines can be accessed without making the dashboard excessively long.

Task Completion Chart

The dashboard includes a task completion chart that displays the number of tasks completed on a particular day. This gives users a visual understanding of productivity and task completion trends.

Dashboard Search

A search bar has been implemented on the dashboard to make information easier to find.

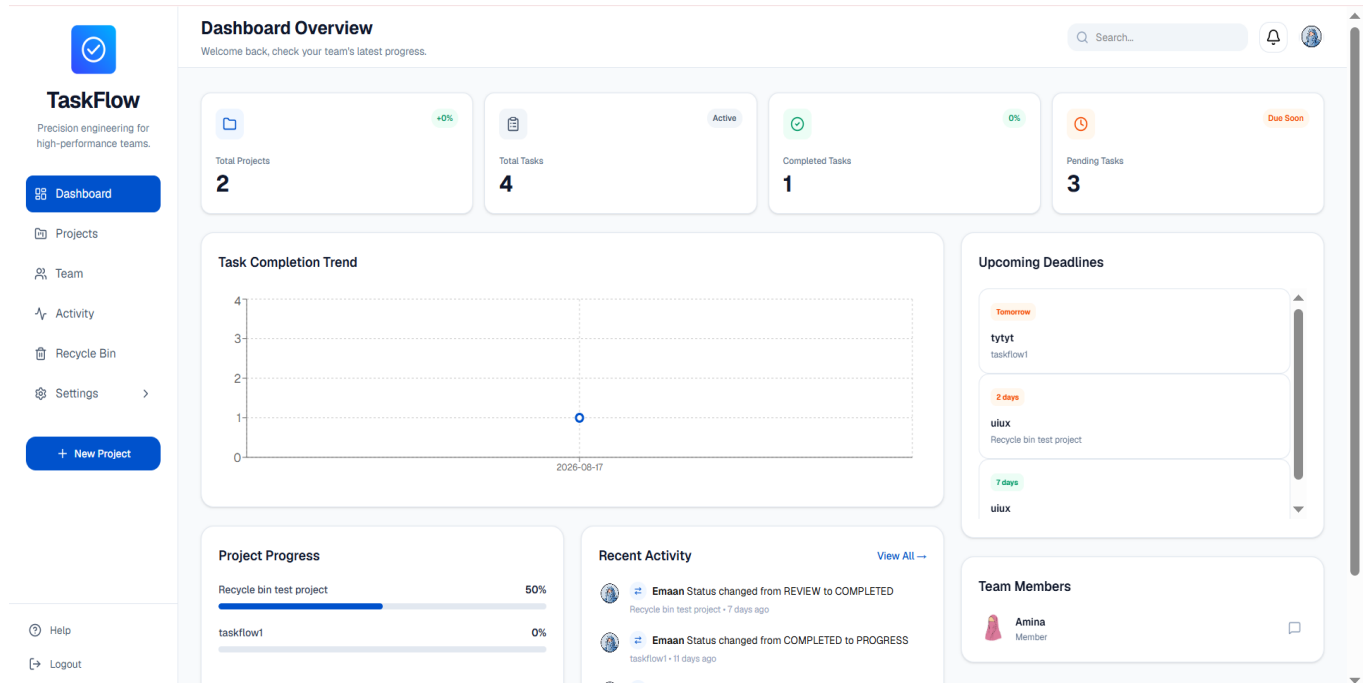


Figure 4: Dashboard Page showing the team members, recent activities, project progress, deadlines, task completion chart, and projects/task statistics.

7. Project Management

TaskFlow provides complete project CRUD functionality.

CRUD stands for:

- Create
- Read
- Update
- Delete

Users can create and manage projects through the Projects section.

Project Creation

Users can create projects by providing the required project information.

After creation, the project becomes available in the user's project workspace.

Project Updating

Project owners have permission to update project information.

Project Deletion

Project owners have permission to delete projects. TaskFlow also implements soft deletion for projects. Instead of immediately removing the project from the database, it can be moved into a

deleted state, allowing the system to maintain the record and support recovery-related functionality.

Project Permissions

Project ownership determines project-level control.

The project owner has full control over project management operations.

This includes:

- Updating the project
- Deleting the project
- Managing task assignments

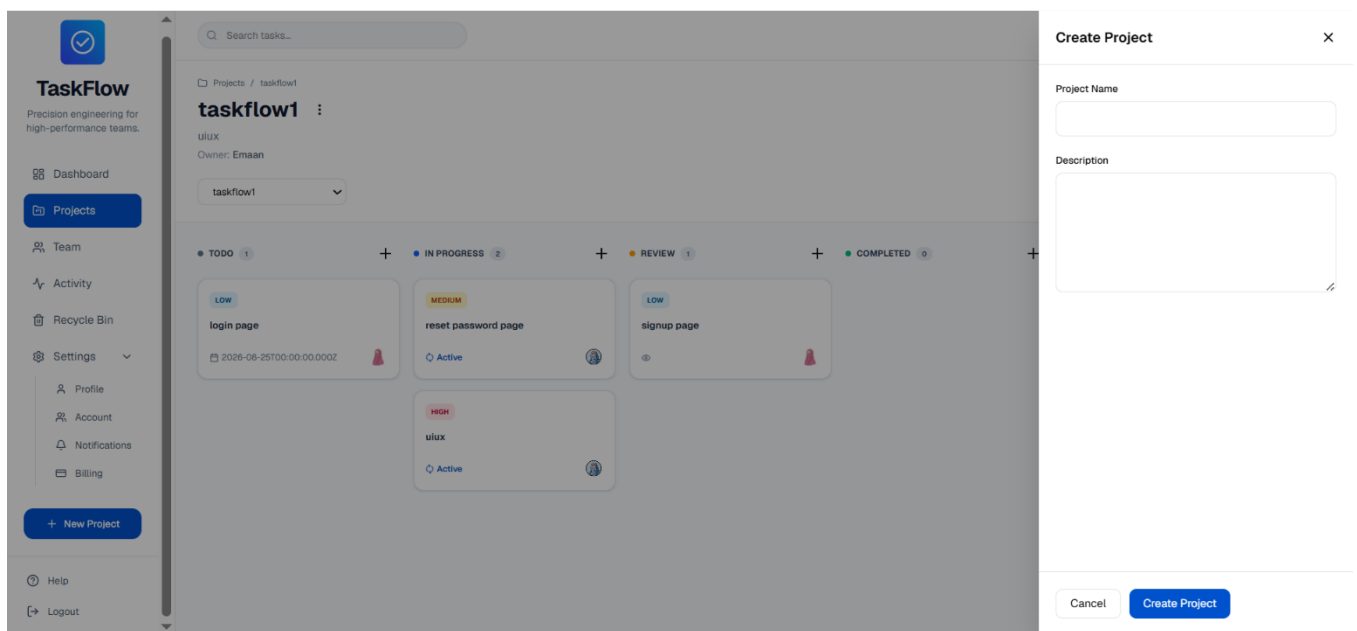


Figure 5: Project drawer to create, update and delete a project

8. Task Management

TaskFlow provides complete task management inside projects.

Each task can contain information such as:

- Title
- Description
- Status
- Due date
- Assignee

- Comments
- Attachments

Task CRUD

Users can:

- Create tasks
- View tasks
- Update tasks
- Delete tasks

Task Assignment

The project owner can assign tasks to project members. The project owner can also assign a task to themselves. This provides flexibility for both team-based and self-managed tasks.

Task Permissions

Task permissions are role-based.

Project Owner

The project owner has full control over tasks.

The owner can:

- Create tasks
- Update tasks
- Delete tasks
- Assign tasks
- Manage task information

Assignee

An assignee can modify tasks assigned to them according to the implemented permissions.

This prevents unauthorized users from modifying tasks that they do not have permission to manage.

9. Kanban Board

TaskFlow uses a Kanban-style workflow for organizing tasks.

The available task states are:

TO DO, IN PROGRESS, REVIEW, COMPLETE.

The Kanban board allows users to understand the current state of every task visually.

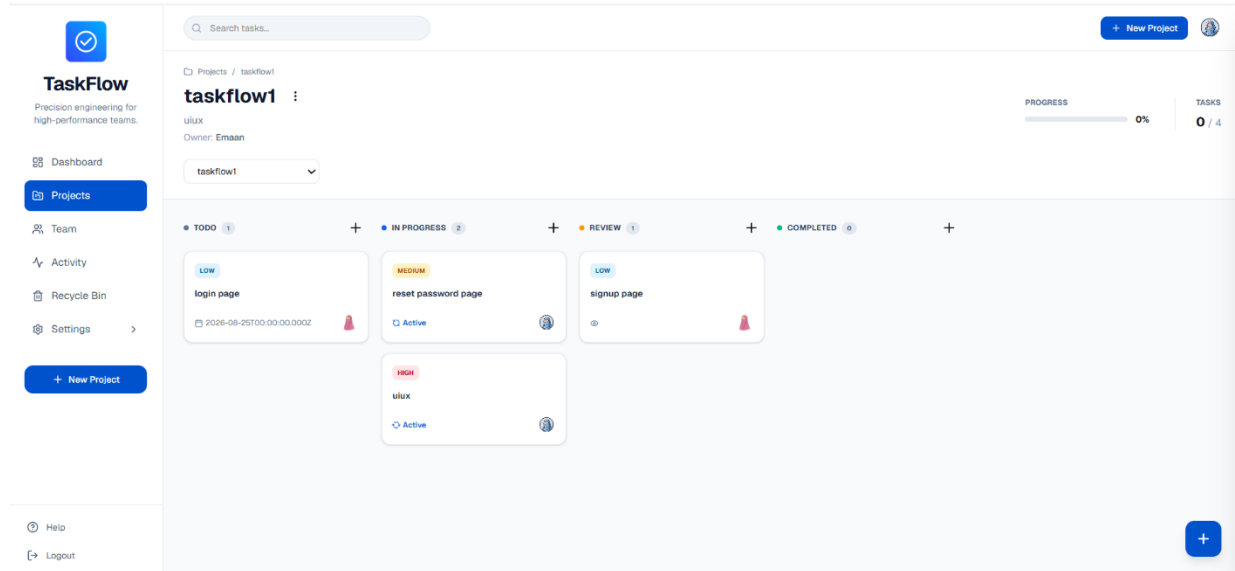


Figure 6: Project page showing the project and their tasks in the kanban

10. Drag-and-Drop Task Management

TaskFlow implements drag-and-drop functionality using DnD Kit.

Users can drag a task from one status column to another.

For example:

TO DO

↓ Drag

IN PROGRESS

↓ Drag

REVIEW

↓ Drag

COMPLETED

11. Task Creation

Tasks can be created from multiple locations within the Kanban board.

Users can create tasks through:

Floating Add Button

A floating plus button is available at the bottom of the Kanban board.

Column Add Buttons

Each status column also contains a plus button.

12. Task Drawer

Task information is displayed inside a task drawer.

The drawer provides access to:

- Task information
- Comments
- Attachments
- Activity timeline
- Task editing
- Task saving

Save Task Loading State

The Save Task button has been improved so that after the user clicks it:

Saving...

is displayed and the button becomes disabled while the operation is in progress.

This prevents:

- Duplicate submissions
- Multiple API requests
- Accidental repeated updates

Task Comments

Users can communicate directly within tasks using comments. Comments provide task-specific communication rather than requiring users to switch to another communication platform.

Task Attachments

TaskFlow provides attachment functionality inside tasks. Users can attach files to tasks and access those attachments through the task interface.

Task Activity Timeline

Each task contains an activity timeline showing relevant task activities. This allows users to understand the history of changes made to a task.

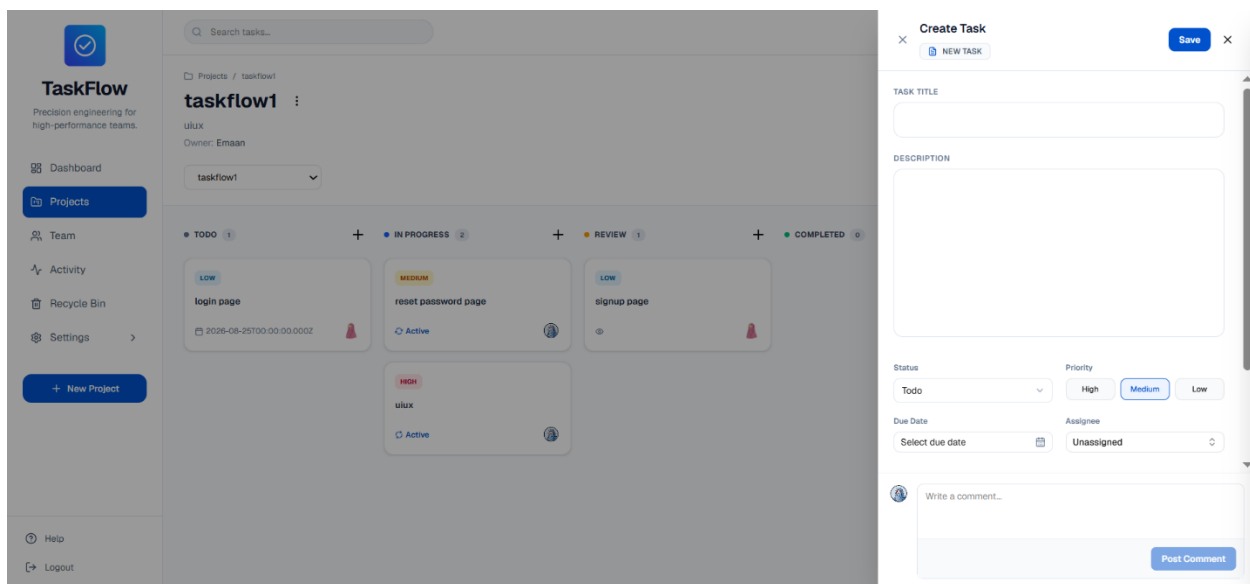


Figure 7: Task drawer to create, update, and delete a task, also to add comments and attachments to the task

13. Activity Page

TaskFlow provides a dedicated Activity page. The Activity page includes frontend and backend implementation. Activity records are displayed using pagination. Instead of loading all activities at once:

Page 1 → Latest 20 activities

Page 2 → Next 20 activities

Page 3 → Next 20 activities

...

This improves usability and reduces unnecessary data loading. Activities can also navigate users to their respective project/task context.

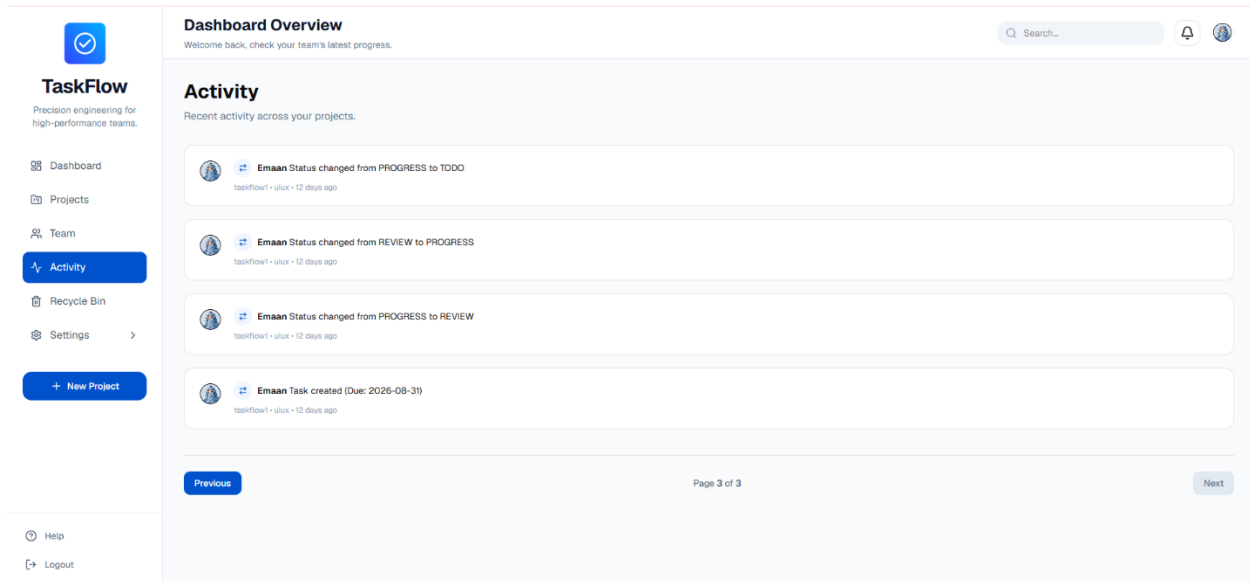


Figure 8: Activity page showing all the activities of the tasks which are owned by the user, or they are part of that task

14. Search Functionality

TaskFlow provides search functionality for tasks in the Projects page. Users can search for tasks instead of manually scanning through all Kanban columns. Dashboard search functionality has also been implemented.

15. Team Management

TaskFlow provides team collaboration functionality.

Users can work with team members by:

- Adding members to projects
- Assigning tasks
- Viewing member profiles
- Inviting members
- Collaborating through comments
- Tracking member-related activity

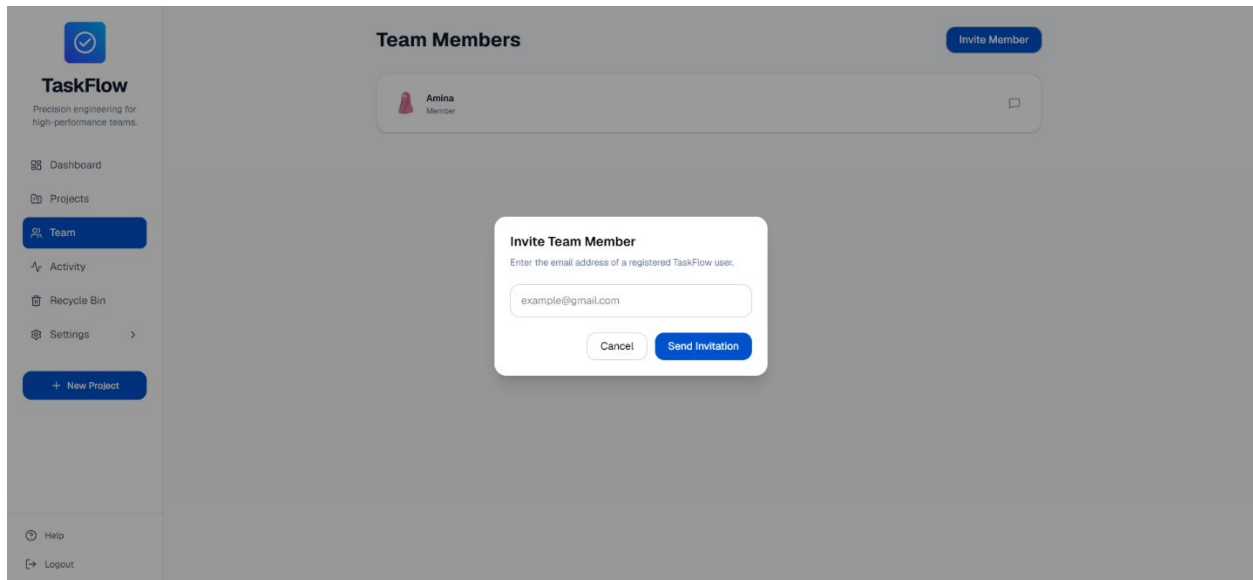


Figure 9: Team page where the user can invite a new team member and can view all the team members on this page

16. Invitation System

TaskFlow includes an invitation system for bringing users into projects. The implemented invitation functionality supports the application's in-app collaboration workflow. The invitation system has been developed as part of the team/project collaboration features.

17. Notifications

TaskFlow includes an in-app notification system.

Notifications are generated for relevant updates made to projects and tasks owned by the user.

Notification Features

Users can:

- View notifications
- See unread notification count
- Mark all notifications as read
- Delete notifications

This allows users to immediately know when new activity requires their attention.

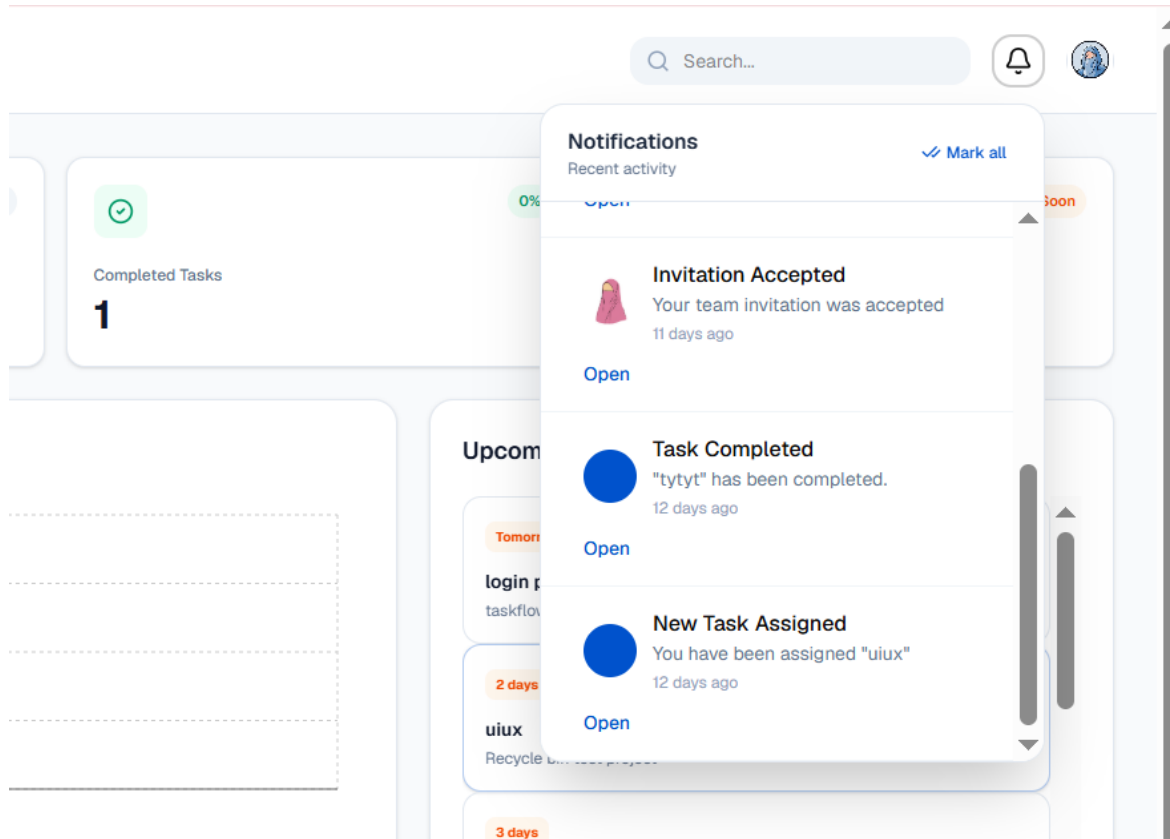


Figure 10: Notification icon on the dashboard page notifies about project and task changes and invitation status

18. User Profile Management

The Settings section provides profile-management functionality.

Users can:

- Update profile information
- View profile information
- Change profile image
- Delete profile image

Other users can view profiles in read-only mode where applicable.

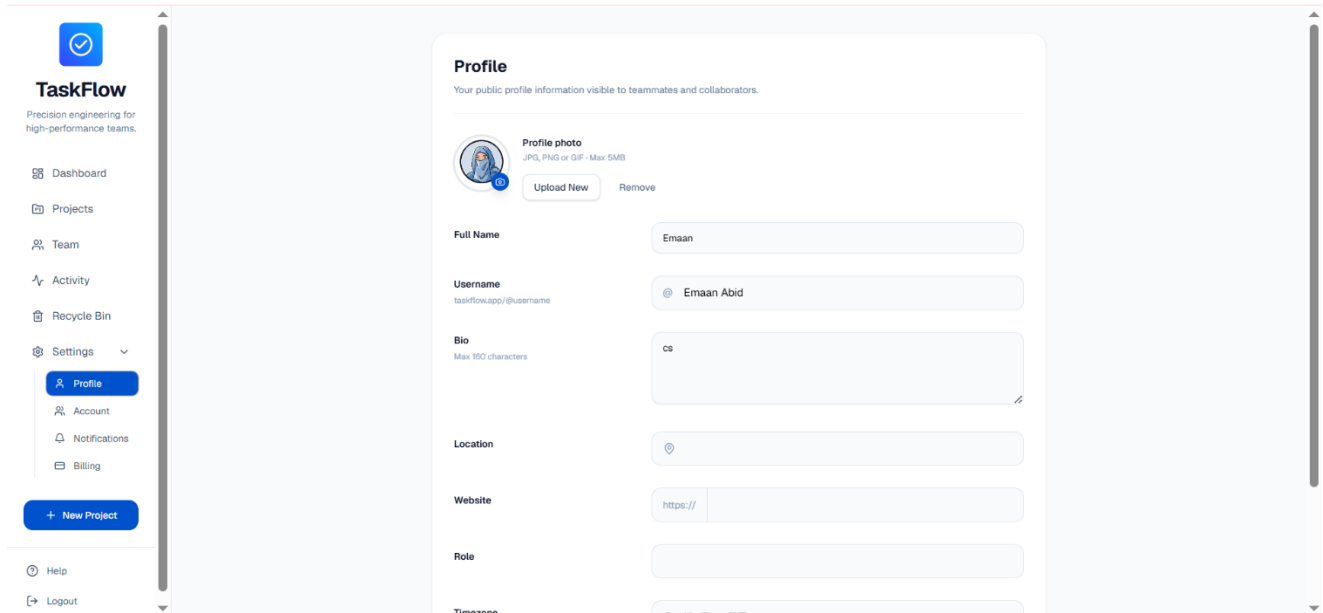


Figure 11: Profile page of the user to update their profile

19. Account Management

TaskFlow provides account-level controls.

Users can update their account email. Users can change their account password. Users can enable 2FA for additional account security. Users can permanently delete their account.

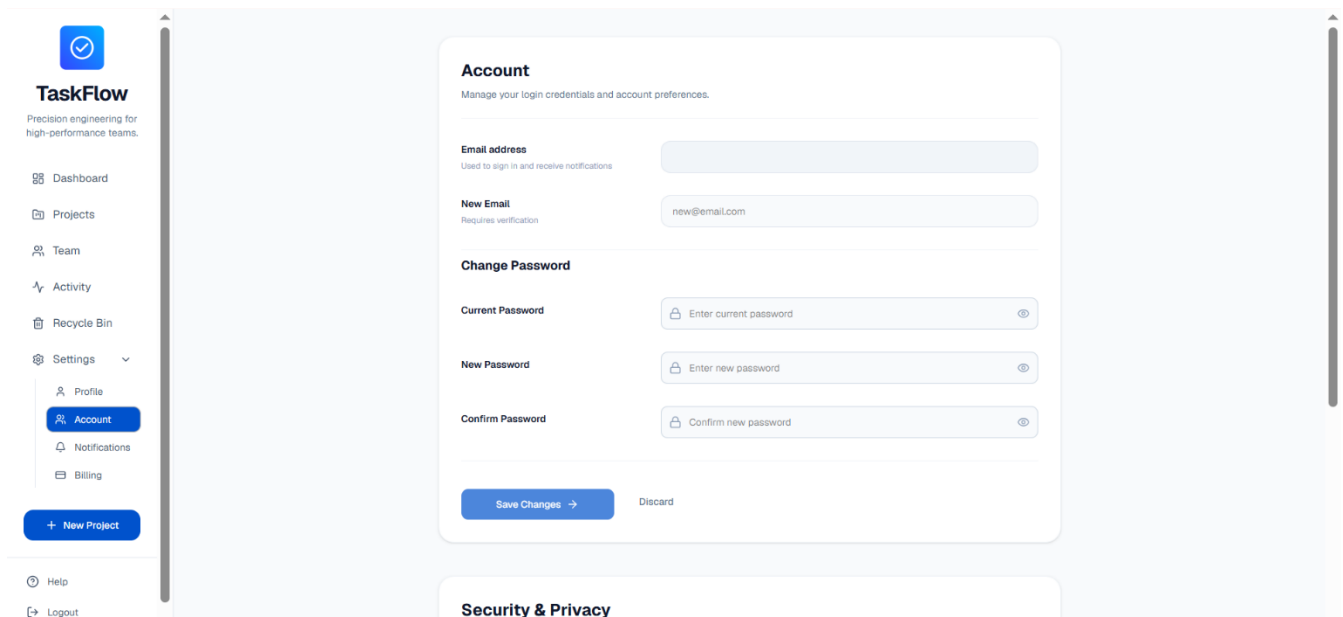


Figure 12: Account page to update user email or password, to enable 2FA, or delete their account permanently

20. Notification Management

Users can control their notification preferences.

Notification settings include categories related to:

- Activity
- Projects
- System notifications

Users can control notification delivery through:

- Email
- Push notifications

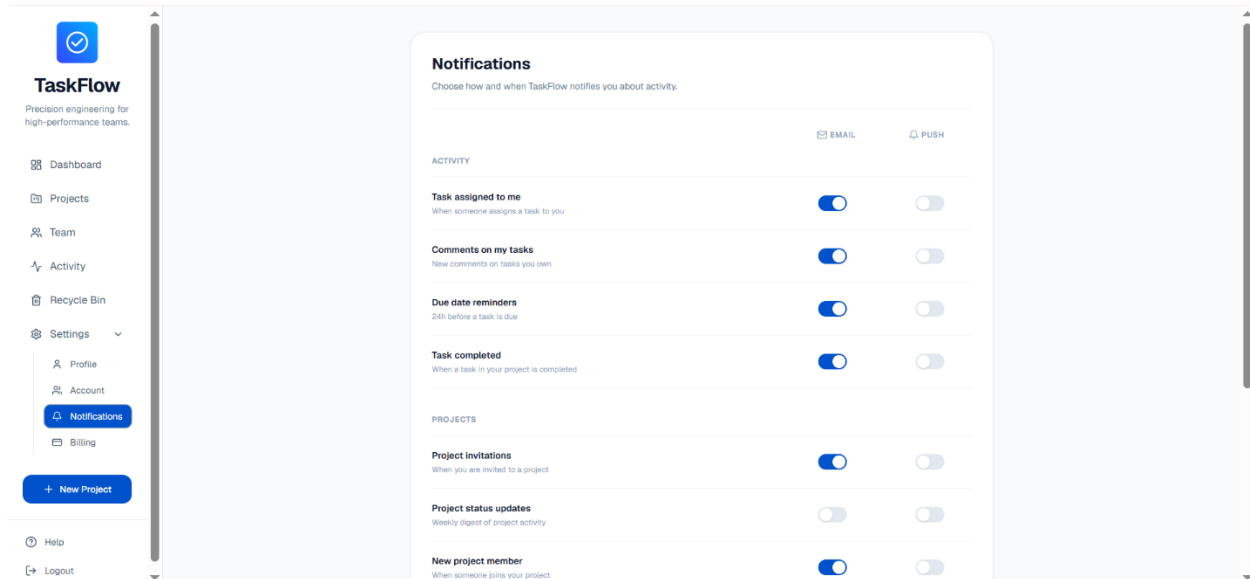


Figure 13: Notification page to manage the notifications

21. Billing Management

TaskFlow includes billing and subscription management. The Billing section provides information about the user's current subscription.

The system supports plans including:

Plan	Monthly Price
FREE	\$0
PRO	\$12
ENTERPRISE	\$39

Billing Features

The billing system provides:

- Current plan
- Plan selection
- Plan updates
- Renewal information
- Payment method management
- Invoice history
- Invoice status
- Billing cycle

Payment Method

The system stores only safe payment metadata such as:

- Card Brand
- Last 4 Digits
- Expiry

Sensitive information such as:

- Full Card Number
- CVV

is not stored by the TaskFlow backend.

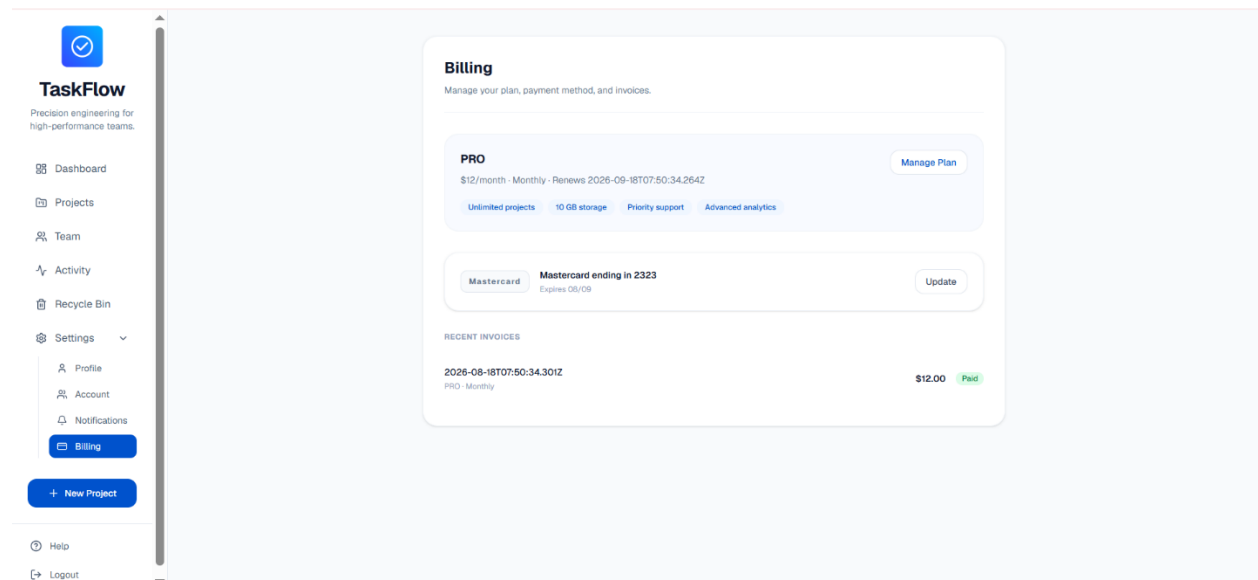


Figure 14: Billing page to manage the bills and subscriptions

22. Recycle Bin / Soft Delete

TaskFlow implements soft deletion for projects.

Instead of immediately deleting a project from the database, the system maintains its record in a deleted state.

This approach provides safer deletion behavior and allows deleted data to be handled separately.

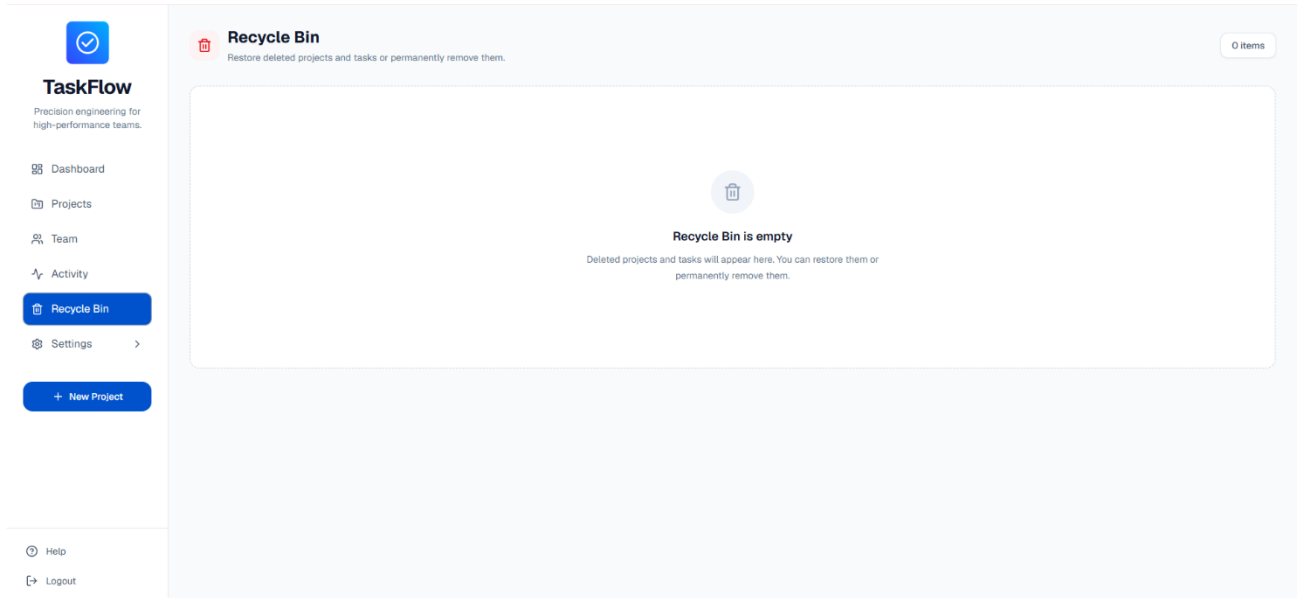


Figure 15: Recycle Bin page which shows the deleted projects and tasks that can be restore or permanently delete

23. Backend Architecture

The backend is implemented using a layered architecture.

The architecture separates responsibilities between different layers.

HTTP Request → Routes → Middleware → Controller → Service → Repository → PostgreSQL

Middleware Layer

The middleware layer handles cross-cutting functionality such as:

- Authentication
- Request validation
- Error handling
- Other request-level processing

Controller Layer

Controllers are responsible for:

- Receiving HTTP requests
- Extracting request information
- Calling services
- Returning HTTP responses

Controllers do not contain the application's main business logic.

Service Layer

The service layer contains business logic.

For example, billing operations such as:

- Plan management
- Pricing
- Renewal calculations
- Payment metadata updates
- Invoice creation

are handled by the service layer.

Repository Layer

The repository layer handles database interaction.

This separation prevents database queries from being unnecessarily mixed with controllers and business logic.

Global Error Handling

Unhandled errors are passed to a centralized error-handling middleware.

This provides standardized API error responses.

24. Error Handling

TaskFlow includes standardized backend error handling.

The purpose of centralized error handling is to provide consistent responses for:

- Validation errors
- Authentication errors
- Authorization errors
- Not-found errors
- Database errors
- Unexpected server errors

Instead of every controller handling errors independently, errors can be mapped and passed to the global error handler.

25. Database

TaskFlow uses PostgreSQL as its relational database and Prisma ORM as the database access layer.

The database is responsible for storing application data related to users, projects, tasks, collaboration, notifications, billing, authentication, and system activities.

Major database entities include:

- Users
- Projects
- Tasks
- Project Members
- Invitations
- Comments
- Attachments
- Activities
- Notifications
- Notification Settings
- Billing
- Invoices
- Token Blacklist

The project uses CUID string identifiers for application entities.

Database Entity Relationships

The major relationships between the entities are:

- A User can own multiple Projects.
- A Project belongs to one owner.
- A Project can contain multiple Tasks.
- A Task belongs to one Project.
- A Task can be assigned to a User.

- Users can participate in multiple Projects through project membership.
- A Project can have multiple members.
- A User can create multiple Comments.
- A Task can contain multiple Comments.
- A Task can have multiple Attachments.
- Projects and Tasks can generate Activity records.
- A User can receive multiple Notifications.
- Each User has notification preference settings.
- A User can have billing/subscription information.
- Billing records can have associated invoices.
- Authentication-related token information can be maintained through the token blacklist.

Database ERD

The following ERD represents the main entities and their relationships:

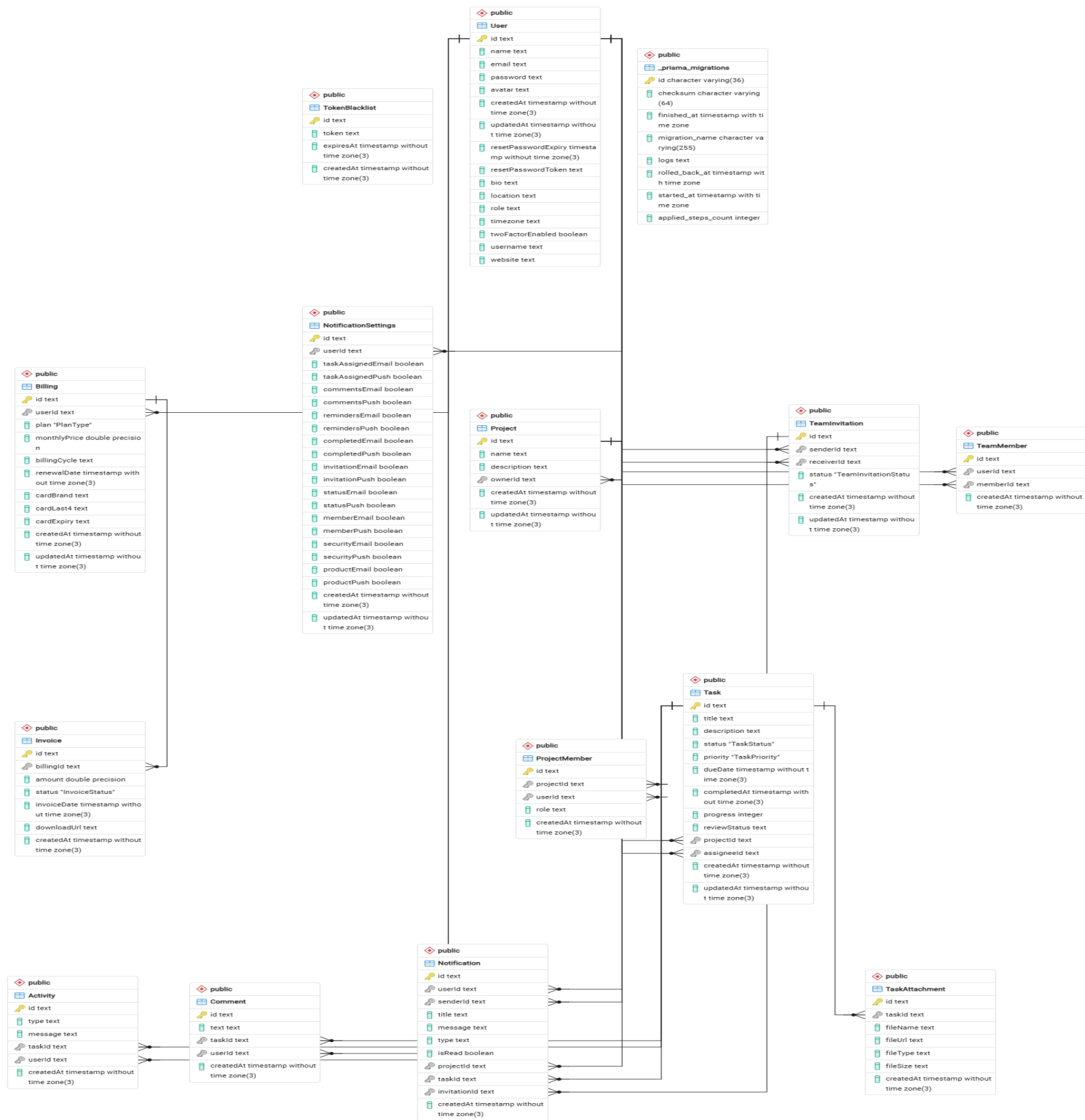


Figure 16: Entity Relationship Diagram of the TaskFlow database

26. Redis Caching

Redis caching has been implemented to improve performance for frequently accessed data.

Caching has been applied to areas including:

- Dashboard
- Projects
- Tasks

The general architecture is:

Frontend → Backend API → Check Redis Cache

If Cache Hit → Return Cached Data.

If Cache Miss → PostgreSQL → Store in Redis → Return Data.

Caching reduces unnecessary database queries for frequently accessed information.

27. API Documentation

TaskFlow provides Swagger API documentation.

Swagger is available at: <http://localhost:5000/api-docs>

The API documentation includes:

- Endpoint descriptions
- Request information
- Response examples
- Parameter explanations
- Possible error responses

This makes the backend easier to understand, test, and integrate.

28. API Summary

TaskFlow provides a RESTful backend API implemented using Node.js, Express.js, and TypeScript.

The API is responsible for authentication, user management, projects, tasks, collaboration, notifications, billing, and other application functionality.

Protected endpoints require a valid JWT access token.

The following table summarizes the major API operations implemented in TaskFlow.

Method	Endpoint	Authentication / Role	Purpose
POST	/api/auth/signup	Public	Creates a new user account
POST	/api/auth/login	Public	Authenticates a user and provides an authentication token
POST	/api/auth/logout	Authenticated	Logs the user out and handles token invalidation
POST	/api/auth/forgot-password	Public	Starts the password recovery process
POST	/api/auth/reset-password	Public / Token	Resets the user's password
GET	/api/users/profile	Authenticated	Retrieves the authenticated user's profile
PATCH	/api/users/profile	Authenticated	Updates profile information
GET	/api/notifications	Authenticated	Retrieves notification settings
PATCH	/api/notifications	Authenticated	Updates notification settings
GET	/api/notifications/unread-count	Authenticated	Retrieves the number of unread notifications
GET	/api/recycle-bin	Authenticated	Retrieves deleted projects/tasks available to the user
GET	/api/projects	Authenticated	Retrieves accessible projects
POST	/api/projects	Authenticated	Creates a project
PATCH	/api/projects/:id	Project Owner	Updates project information
DELETE	/api/projects/:id	Project Owner	Soft-deletes a project
GET	/api/projects/:id/tasks	Project Member	Retrieves tasks belonging to a project
POST	/api/projects/:id/tasks	Project Owner	Creates a task
PATCH	/api/tasks/:id	Owner / Assignee	Updates an authorized task
DELETE	/api/tasks/:id	Project Owner / Authorized User	Deletes a task according to implemented permissions
POST	/api/tasks/:id/comments	Project Member	Adds a comment to a task
GET	/api/tasks/:id/comments	Project Member	Retrieves task comments
GET	/api/activity	Authenticated	Retrieves activity records
GET	/api/billing	Authenticated	Retrieves billing/subscription information

API Request Flow

The API request flow is:

Frontend → Axios Request → JWT Authorization Header → Express Router → Authentication Middleware → Authorization / Validation → Controller → Service → Repository / Prisma → PostgreSQL → Response → Frontend

29. Environment Configuration

TaskFlow provides “.env.example” files so that developers can understand which environment variables are required.

The basic workflow is:

```
cp .env.example .env
```

Windows Command Prompt:

```
copy .env.example .env
```

Windows PowerShell:

```
Copy-Item .env.example .env
```

The developer then updates the environment variables according to their local setup.

30. Security and Authentication

Security is an important part of TaskFlow because the application handles user accounts, project information, task data, notifications, and billing-related metadata.

The system uses multiple security mechanisms including JWT authentication, token handling, authorization, password management, two-factor authentication, backend validation, and secure payment metadata handling.

Authentication Flow

The authentication flow is:

User → Login Form → Backend Authentication API → Validate Credentials → Verify Password → Generate JWT → Frontend Stores Authentication Token → Token Attached to Protected Requests → Authentication Middleware → JWT Verification → Authenticated User Identified → Authorized Resource Access

The JWT allows the backend to identify the authenticated user for protected API requests.

JWT Authentication

TaskFlow uses JSON Web Tokens for authentication.

After successful login:

1. The user submits their credentials.
2. The backend validates the credentials.
3. A JWT is generated.
4. The frontend stores the authentication token.
5. The token is attached to protected API requests.
6. The backend authentication middleware verifies the token.
7. The authenticated user's identity is made available to the request.
8. Authorization checks determine whether the user can perform the requested operation.

Protected resources therefore require a valid authentication token.

Authorization

Authentication determines:

Who is the user?

Authorization determines:

What is the user allowed to do?

TaskFlow applies authorization checks to project and task operations.

For example:

JWT → User ID → Project / Task → Determine User Role → Check Permission → Allow or Reject

This prevents unauthorized users from performing restricted project or task operations.

Logout and Token Handling

Logout is implemented on both the frontend and backend.

The frontend removes the authentication state/token used for subsequent API requests.

The backend also handles logout-related token invalidation using the implemented token handling mechanism.

The project database includes a Token Blacklist entity for managing invalidated tokens. This helps prevent a previously issued token from remaining usable after logout when token invalidation is required.

Password Management

TaskFlow provides:

- Secure login
- Password change
- Forgot password functionality
- Password reset functionality

Password-related operations are handled by the backend rather than trusting the frontend.

Two-Factor Authentication

TaskFlow provides Two-Factor Authentication (2FA) as an additional account security mechanism. The purpose of 2FA is to provide an additional verification layer beyond the user's password.

Users can enable 2FA through account settings, but right now only the toggle button works and saves the state in the backend also.

Backend Validation

Backend validation is used to prevent invalid data from being silently processed.

Validation is applied to incoming requests before business operations are performed.

The backend also uses centralized error handling for:

- Invalid requests
- Authentication failures
- Authorization failures
- Resource-not-found errors
- Database errors
- Unexpected server errors

Secure Payment Metadata

TaskFlow does not store sensitive card information such as:

- Full card number
- CVV

The system stores only non-sensitive payment metadata such as:

- Card brand
- Last four digits
- Expiry information

This reduces the amount of sensitive payment information maintained by the application.

31. Role-Based Access Control and Permissions

TaskFlow implements role-based access control to ensure that users can only perform operations allowed by their role within a project.

The main permission categories are:

- **Project Owner**
- **Assignee**
- **Team Member**

Permissions are applied at the project and task levels.

Project Owner

The Project Owner has the highest level of project-level access.

The owner can:

- Manage project information
- Manage project members
- Assign tasks
- Update tasks
- Delete tasks
- Manage project deletion and recovery
- Manage project-related settings

Assignee

An Assignee is a project member who has been assigned a particular task.

The assignee can access and modify the task according to the application's task-level permissions as represented in the below permission matrix table.

Team Member

A Team Member can access projects they belong to and participate in permitted collaboration activities.

Permissions Matrix

Operation	Project Owner	Assignee	Team Member
View project	✓	✓	✓
View project tasks	✓	✓	✓
Create project	✓	—	—
Update project	✓	—	—
Delete project	✓	—	—
Restore deleted project	✓	—	—
Permanently delete project	✓	—	—
Create task	✓	—	—
View task	✓	✓	✓
Update assigned task	✓	✓	—
Update another user's task	✓	—	—
Delete task	✓	Only the task assigned to them	—
Assign task	✓	—	—
Self-assign task	✓	—	—
Add task comment	✓	✓	✓
View task comments	✓	✓	✓
Add task attachment	✓	✓	—
View activity	✓	✓	✓
Manage project members	✓	—	—
Invite project members	✓	—	—
View own notifications	✓	✓	✓
Manage own notification settings	✓	✓	✓
Manage own profile	✓	✓	✓
Manage own account	✓	✓	✓
Manage billing	✓	—	—

Authorization Principle

Authorization is enforced on the backend rather than relying only on frontend UI restrictions.

The general authorization process is:

User Request → JWT Verification → Identify User → Identify Project / Task → Check User Role → Check Permission → Allow / Reject Operation.

This prevents users from bypassing frontend restrictions by directly calling the API.

32. Frontend Technology Stack

The frontend uses:

Technology	Purpose
React	UI development
TypeScript	Type safety
Vite	Development/build tooling
Tailwind CSS	Styling
React Router	Client-side routing
Axios	API communication
DnD Kit	Drag-and-drop
Recharts	Charts and analytics
Lucide React / React Icons	UI icons

33. Backend Technology Stack

The backend uses:

Technology	Purpose
Node.js	Runtime
Express.js	REST API
TypeScript	Type safety
Prisma	ORM/database access
PostgreSQL	Relational database
JWT	Authentication
Swagger	API documentation
Redis	Caching

34. System Architecture

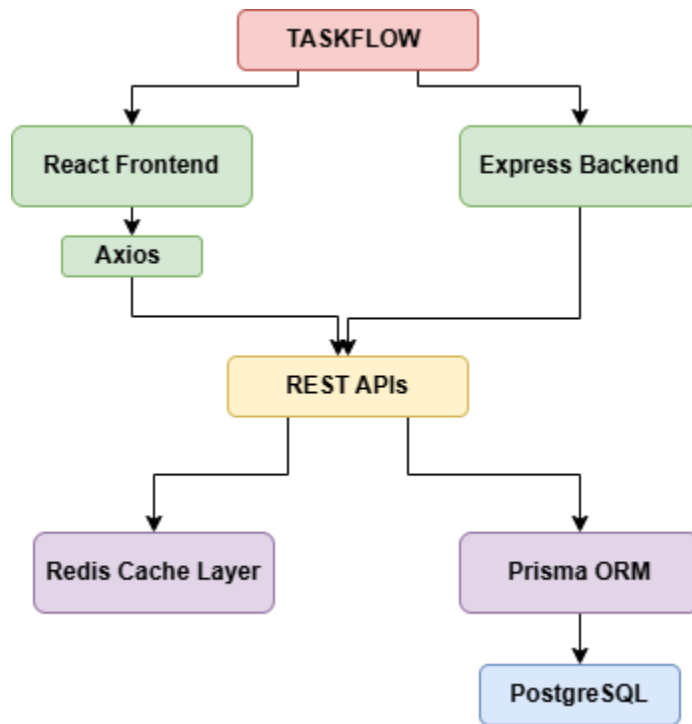


Figure 17: The complete system architecture of the TaskFlow

35. Complete Application Workflow

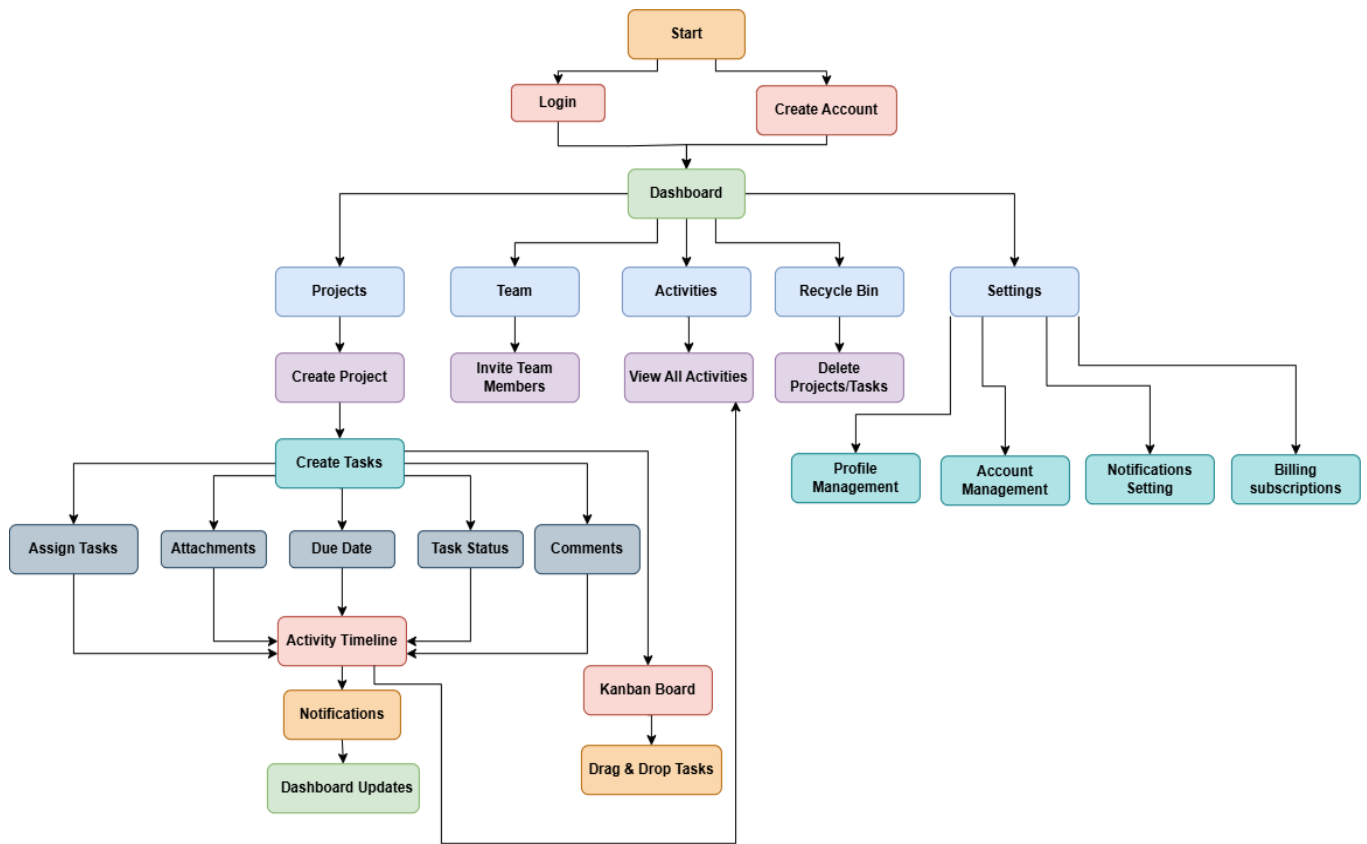


Figure 18: User workflow of the TaskFlow

36. Functional Requirements

- **Authentication:** Offers complete security workflows including Signup, Login, Logout, Password Reset, JWT Authentication, and Two-Factor Authentication (2FA).
- **Projects:** Supports full lifecycle management with Create, View, Update, Delete, Soft Delete, Project Permissions, and Progress tracking.
- **Tasks:** Provides detailed task handling with Create, View, Update, Delete, Assign Task, Self Assignment, Comments, Attachments, Activity Timeline, Search, Drag & Drop, and Task Statuses.
- **Dashboard:** Delivers high-level operational insights via Project Statistics, Task Statistics, Team Members overview, Recent Activity feeds, Deadlines, Progress metrics, Completion Charts, and Search.
- **Account:** Enables user profile control through Profile Management, Profile Image uploads, Email Updates, Password Changes, 2FA settings, and Account Deletion.
- **Notifications:** Keeps users updated using In-App Notifications, Unread Count indicators, Mark All as Read options, and Delete Notifications.

- **Billing:** Handles subscription workflows including Current Plan details, Plan Selection, Payment Method management, Invoice History, Renewal Information, and Subscription Plans.

37. Non-Functional Requirements

- **Performance:** Uses Redis caching for high-frequency dashboard, project, and task data, along with pagination for activity logs to streamline data retrieval.
- **Scalability:** Enforces a clean separation between Controllers, Services, Repositories, and the Database to simplify backend expansion and lower primary database load.
- **Maintainability:** Built with TypeScript, a layered backend architecture, reusable React components, Service/API separation, centralized error handling, and environment configuration.
- **Usability:** Enhances user experience through a Kanban workflow, drag-and-drop mechanics, responsive UI, search capabilities, dashboard analytics, notifications, a task drawer, and profile management.

38. Responsive Design

TaskFlow provides a responsive user interface so that the application's interface can adapt to different screen sizes.

39. External/Supporting Services

TaskFlow primarily uses its own React frontend, Express/TypeScript backend, PostgreSQL database, Prisma ORM, and Redis caching infrastructure.

The production deployment uses:

- Vercel for frontend hosting.
- Railway for backend hosting.
- Railway PostgreSQL for the production database.
- Redis for application caching.
- Bee Keeper for data visualization.

40. Installation Requirements

Before running TaskFlow locally, the following software is required:

- Node.js v18
- npm

- PostgreSQL v14

Recommended hardware

- Minimum 4 GB RAM
- At least 1 GB free storage
- Modern dual-core processor

Supported OS

- Windows 10/11
- Linux
- MacOS

41. Local Installation

Step 1: Clone Repository

git clone <https://github.com/EmaanAbid2711/TaskFlow-Frontend.git>

Navigate into the project: cd TaskFlow-Frontend

Project structure:

TaskFlow-Frontend

```
└── taskflow
    ├── frontend
    └── backend
```

Step 2: Backend Setup

Navigate to: cd taskflow/backend

Install dependencies:

npm install

Create environment file.

For Linux/macOS: cp .env.example .env

For Windows CMD: copy .env.example .env

For Windows PowerShell: Copy-Item .env.example .env

Configure the environment variables.

Create a PostgreSQL database: taskflow_db, or use another database name and update the DATABASE_URL

Run: `npm run setup`

Then: `npm run dev`

The backend will run on `http://localhost:5000`

Step 3: Frontend Setup

Open another terminal.

Navigate to: `cd taskflow/frontend`

Install dependencies:

`npm install`

Create the environment file: `cp .env.example .env`

On Windows CMD: `copy .env.example .env`

On PowerShell: `Copy-Item .env.example .env`

Configure the frontend environment variables.

Start the frontend: `npm run dev`

Frontend will run on: `http://localhost:5173`

Step 4: Running the Complete Application

Two terminals are required.

Terminal 1 — Backend

`cd taskflow/backend`

`npm install`

`npm run setup`

`npm run dev`

Terminal 2 — Frontend

`cd taskflow/frontend`

`npm install`

`npm run dev`

Open: `http://localhost:5173`

Step 5: API Documentation

After starting the backend, Swagger documentation is available at: `http://localhost:5000/api-docs`

Swagger provides an interface for:

- Viewing endpoints

- Understanding request parameters
- Viewing request bodies
- Viewing response structures
- Testing APIs
- Understanding possible errors

42. Deployment

TaskFlow is deployed as a full-stack production application.

Frontend is hosted on Vercel. The backend is hosted on Railway, and the database is hosted on Railway PostgreSQL.

The frontend communicates directly with the deployed backend.

43. Live Application

Based on the deployment information provided:

Service	Platform	URL
Frontend	Vercel	https://task-flow-frontend-woad.vercel.app
Backend	Railway	https://taskflow-backend-production-df3c.up.railway.app
API Documentation	Railway	https://taskflow-backend-production-df3c.up.railway.app/api-docs

44. Testing and Validation

TaskFlow was tested during development and after deployment to verify that the major functional and security requirements were working correctly.

Testing focused on:

- Authentication
- Authorization
- Projects
- Tasks
- Kanban workflow
- Collaboration
- Notifications

- Dashboard
- Billing
- Account management
- Recycle Bin
- API communication
- Production deployment

Test Case Format

Each test case contains:

- Test Case ID
- Test Area
- Test Scenario
- Expected Result
- Actual Result
- Status

Functional Test Cases

ID	Test Area	Test Scenario	Expected Result	Actual Result	Status
TC-01	Signup	Register with valid information	New account is created successfully	Account was created	Pass
TC-02	Signup	Register with invalid/duplicate information	Validation/error message is displayed	Validation was displayed	Pass
TC-03	Login	Login with valid credentials	User is authenticated and redirected to application	User was authenticated	Pass
TC-04	Login	Login with invalid credentials	Login is rejected	Login was rejected	Pass

TC-05	Logout	Click logout	User is logged out and authentication state is cleared	Logout completed successfully	Pass
TC-06	Password Reset	Request password recovery	Password recovery workflow is initiated	Request was processed	Pass
TC-07	Project	Create project	Project is created and displayed	Project was created	Pass
TC-08	Project	Update project as owner	Project information is updated	Project was updated	Pass
TC-09	Project	Unauthorized user attempts restricted project operation	Operation is rejected	Operation was rejected	Pass
TC-10	Task	Create task	Task appears in selected project	Task was created	Pass
TC-11	Task	Assign task to member	Selected member becomes assignee	Assignment was successful	Pass
TC-12	Task	Assign task to self	Current user becomes assignee	Self-assignment worked	Pass
TC-13	Kanban	Drag task between columns	Task status changes to destination column	Status changed successfully	Pass
TC-14	Task	Edit assigned task	Authorized changes are saved	Changes were saved	Pass
TC-15	Comment	Add task comment	Comment appears on task	Comment was added	Pass
TC-16	Attachment	Upload task attachment	Attachment becomes available in task	Attachment was uploaded	Pass
TC-17	Activity	Open Activity page	Activities are displayed	Activities were displayed	Pass
TC-18	Activity	Navigate between activity pages	Correct activity page is loaded	Pagination worked	Pass

TC-19	Search	Search for a task	Matching task results are displayed	Search returned matching results	Pass
TC-20	Dashboard	Open dashboard	Statistics and project information are displayed	Dashboard loaded correctly	Pass
TC-21	Notification	View notifications	User's notifications are displayed	Notifications were displayed	Pass
TC-22	Notification	Mark notifications as read	Notifications become read	Notifications were marked read	Pass
TC-23	Notification Settings	Change notification preference	Preference is saved	Preference was saved	Pass
TC-24	Profile	Update profile information	Profile information is updated	Profile was updated	Pass
TC-25	Account	Change password	Password is updated successfully	Password was changed	Pass
TC-26	Billing	View current subscription	Current billing information is displayed	Billing information was displayed	Pass
TC-27	Recycle Bin	Soft-delete project	Project moves to deleted state/Recycle Bin	Project was soft-deleted	Pass
TC-28	Recycle Bin	Restore deleted project	Project becomes available again	Project was restored	Pass
TC-29	Recycle Bin	Permanently delete deleted item	Item is permanently removed	Permanent deletion worked	Pass
TC-30	API	Access protected endpoint without valid authentication	Request is rejected	Unauthorized request was rejected	Pass

Security Test Cases

ID	Test Scenario	Expected Result	Actual Result	Status
----	---------------	-----------------	---------------	--------

SEC-01	Access protected API without JWT	API returns unauthorized response	Unauthorized response returned	Pass
SEC-02	Access protected API with invalid JWT	Request is rejected	Request was rejected	Pass
SEC-03	Project member attempts owner-only operation	Operation is denied	Operation was denied	Pass
SEC-04	User attempts to access unauthorized project/task resource	Access is denied	Access was denied	Pass
SEC-05	Logout and attempt authenticated request using invalidated authentication state	Request should no longer be authorized	Request was rejected	Pass
SEC-06	Submit invalid request data	Backend validation/error handling responds appropriately	Error response was returned	Pass

Performance Validation

Redis caching was also validated for frequently accessed application data.

The following areas were checked:

- Dashboard data
- Project data
- Task data

The expected behavior was:

Request

↓

Check Redis

↓

Cache Hit → Return Cached Data

|

└─ Cache Miss



PostgreSQL



Store Result in Redis → Return Data.

Activity pagination was also tested to ensure that large activity lists are split into manageable pages rather than retrieved as a single large dataset.

Deployment Validation

Production deployment was validated by checking:

- Frontend availability
- Backend API availability
- Frontend-to-backend communication
- Database connectivity
- Protected API requests
- Production API responses
- Swagger documentation

45. Strengths of the System

TaskFlow provides several advantages over a basic task-management application.

1. Centralized Management

Projects, tasks, team members, comments, files, activities, and notifications are available from one platform.

2. Visual Workflow

The Kanban board provides an easy-to-understand representation of task progress.

3. Role-Based Access

Project owners and task assignees have different permissions.

4. Collaboration

Comments, attachments, invitations, activities, and notifications support teamwork.

5. Analytics

Dashboard statistics and charts provide an overview of productivity and project progress.

6. Performance

Redis caching and activity pagination reduce unnecessary data retrieval.

7. Maintainable Backend

The layered backend architecture separates responsibilities.

8. Production Deployment

The application is deployed and accessible through a production frontend and backend.

Limitations

Based on the currently provided implementation, some areas can still be extended in future versions.

The current system can be improved with:

- Advanced project templates
- Calendar integrations
- More sophisticated billing/payment processing

46. Implemented Features and Future Planned Features

Below are the TaskFlow functionality implemented in the current system and functionality planned for future versions.

Currently Implemented Features

The following features are implemented in the current TaskFlow application:

Authentication and Account

- User signup
- User login
- User logout
- JWT authentication
- Password reset
- Password change
- Two-factor authentication
- Profile management
- Profile image management

- Account management
- Account deletion

Project Management

- Project creation
- Project viewing
- Project updating
- Project deletion
- Project permissions
- Project members
- Project progress tracking
- Project soft deletion
- Recycle Bin
- Project restoration
- Permanent deletion

Task Management

- Task creation
- Task viewing
- Task updating
- Task deletion
- Task assignment
- Self-assignment
- Task status management
- Kanban board
- Drag-and-drop
- Task comments
- Task attachments
- Task activity timeline
- Task search

Collaboration

- Team members
- Project invitations
- Comments
- Attachments
- Activity tracking
- In-app notifications

Dashboard

- Project statistics
- Task statistics
- Completed task statistics
- Project progress
- Upcoming deadlines
- Recent activity
- Task completion chart
- Dashboard search

Account and System

- Notification preferences
- Billing and subscription information
- Payment metadata
- Invoice information
- Swagger API documentation
- Automated database setup
- Centralized error handling
- Redis caching
- Activity pagination
- Responsive interface

Deployment

- Production frontend deployment

- Production backend deployment
- Production PostgreSQL database
- Frontend-to-backend production communication

Future Planned Features

The following features are not considered part of the currently implemented core functionality and are planned for future versions.

Email Notifications and Email Invitations

The current system provides in-app notifications and invitations.

A future version can extend this functionality with:

- Email notifications
- Email-based project invitations
- Password recovery email improvements
- Notification email templates
- Configurable email delivery

Advanced Analytics

Future versions can provide:

- Weekly productivity reports
- Monthly productivity reports
- Team performance reports

Calendar Integration

Future versions could integrate TaskFlow with:

- Google Calendar
- Microsoft Outlook Calendar
- Other calendar services

Advanced Search

Search functionality can be expanded to include:

- Users
- Comments
- Activities
- Attachments

External Integrations

Future versions could integrate TaskFlow with external collaboration and development platforms such as:

- GitHub
- Slack
- Microsoft Teams
- Google Calendar

47. Conclusion

TaskFlow is a complete full-stack task and project management platform developed to simplify project organization, task tracking, team collaboration, and productivity monitoring.

The application combines:

- Secure authentication
- Project management
- Task management
- Kanban workflow
- Drag-and-drop
- Task assignment
- Comments
- Attachments
- Activity tracking
- Dashboard analytics
- Notifications
- Invitations
- Profile management
- Account management
- Billing
- Role-based permissions
- Soft deletion
- Redis caching
- Automated database setup

- Standardized error handling
- Swagger documentation
- Production deployment

The system follows a structured architecture where the React frontend communicates with an Express/TypeScript backend through REST APIs. Prisma provides database access to PostgreSQL, while Redis is used to improve performance for frequently accessed information.

The completed system demonstrates the practical application of frontend development, backend development, database management, authentication, authorization, API design, caching, software architecture, responsive UI design, deployment, and collaborative project management.

TaskFlow is therefore not only a task-management interface but a complete full-stack software engineering project demonstrating the development and deployment of a production-oriented web application.